

UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

Authenticity: Message Authentication and Digital Signature

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

Message Authentication

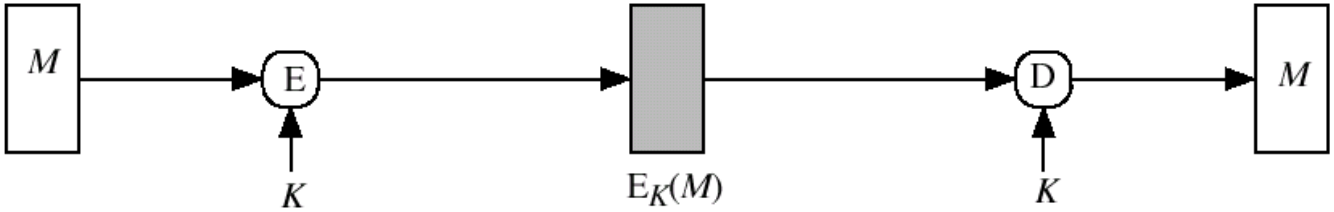
- Message authentication is concerned with:
  - **protecting the integrity of a message**
  - **origin authentication**
    - validating identity of originator
  - **in some cases, also non-repudiation of origin**

- Possible approaches:
  - **symmetric mechanisms (shared secret key)**
    - symmetric encryption
      - sometimes together with an internal integrity check
    - message authentication code
      - keyed one-way functions
  - **asymmetric mechanisms (private/public key pair)**
    - asymmetric encryption
    - digital signature

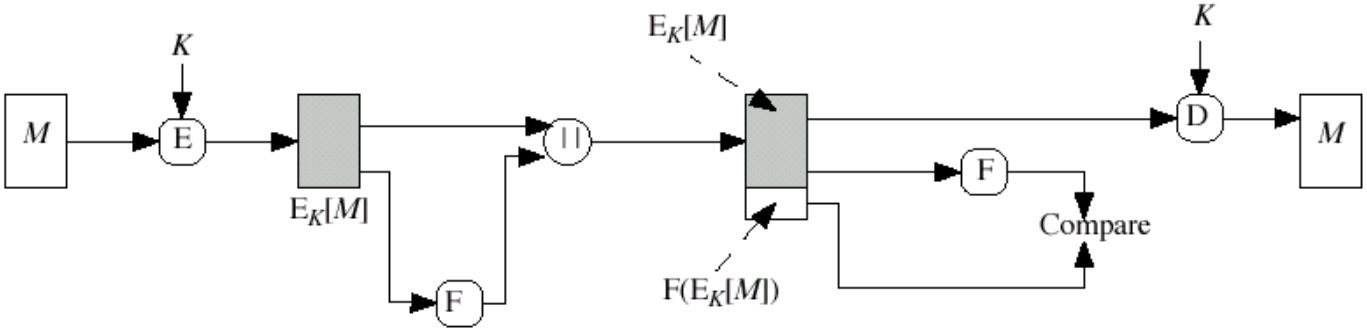
Message Authentication using symmetric keys

Msg. Auth. - Secret-key Encryption

- Symmetric encryption:
  - **encryption may provide both confidentiality and origin authentication**
  - **however, need to recognize corrupted messages**
    - based on the received message or with an explicit internal integrity check (see next slide)

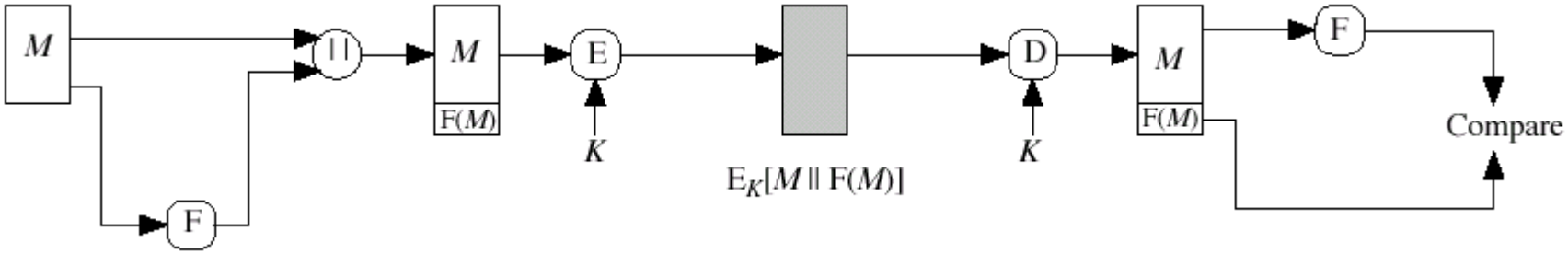


  - **an external error control (checksum) doesn't help**
    - does not securely protect the integrity



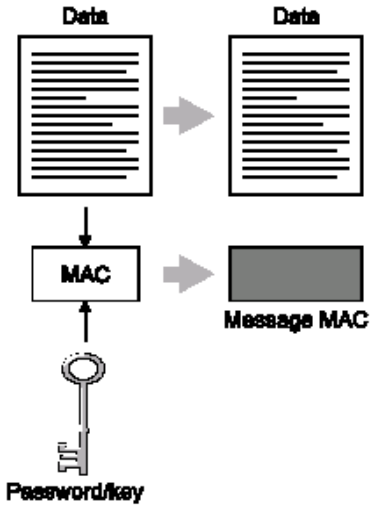
Msg. Auth. - Secret-key Encryption (cont.)

- Internal integrity check, through a manipulation detection code
  - **a sort of robust checksum, e.g. a hash function**
  - **effectiveness depends on the used encryption algorithm**
  - **binds authentication with message encryption**



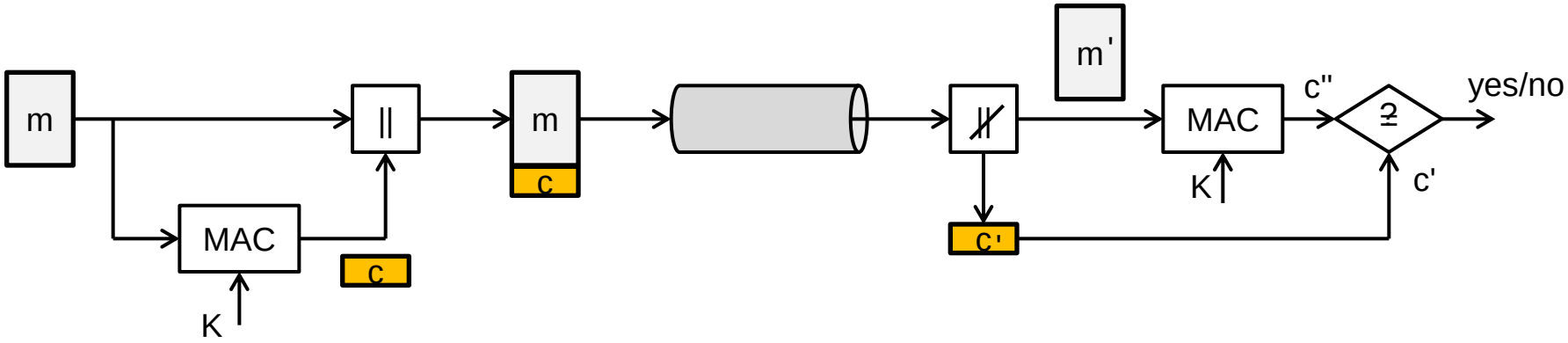
Message Authentication Code (MAC)

- Cryptographic checksum, generated by an algorithm that creates a small fixed-sized block
  - **depending on both message and a secret key K**
    - $MAC = C_K(M) = C(K,M)$
  - **condenses a variable-length message M to a fixed-sized authenticator**
    - it doesn't need to be reversible
    - is a many-to-one function
      - potentially many messages have same MAC
      - but finding these needs to be very difficult



Message Authentication Code (MAC) (cont.)

- Use of MAC for message authentication:
  - **appended to message as a signature**
  - **receiver performs same computation on message and checks it matches the MAC**
    - provides assurance that message is unaltered and comes from sender
  - **can be used also without enforcing confidentiality**



Message Authentication Code (MAC) (cont.)

- MAC functions have to satisfy the following requirements:
  - **knowing a message and MAC, must be infeasible to find another message with same MAC**
  - **it should be also infeasible to find two messages with same MAC**
- Additional properties:
  - **MAC value should be uniformly distributed**
  - **MAC should depend equally on all bits of the message**
- Properties similar to hash functions
  - **in addition, MAC uses an input key**
  - **often seen as keyed hash function**

Message Authentication Code (MAC) (cont.)

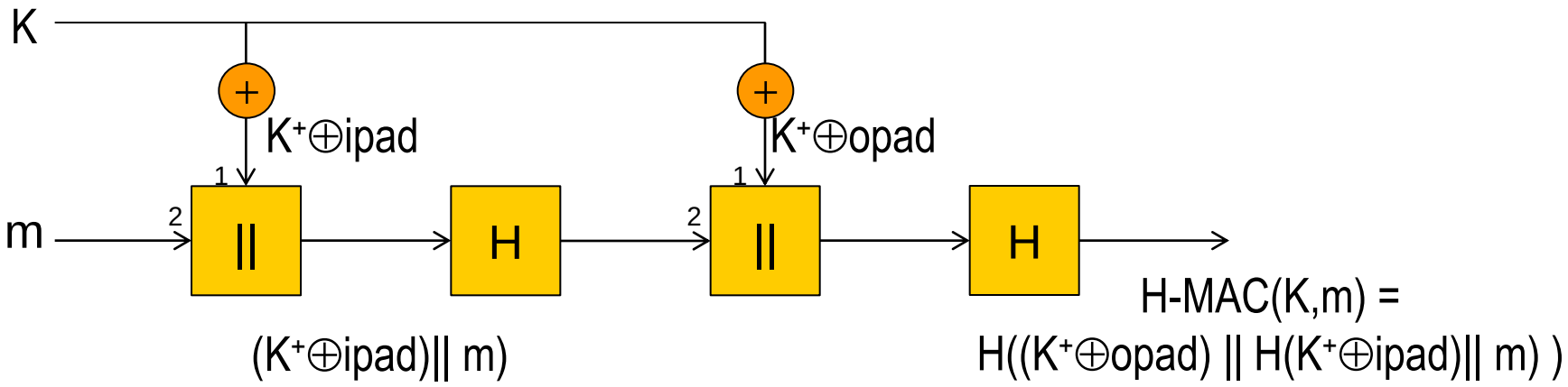
- Advantages:
  - **simplicity**
  - **the message does not need to be encrypted**
    - faster
    - sometimes only authentication is needed
- In case secrecy is also required use, encryption can be applied separately
  - **using two separate keys**
  - **sometimes need authentication to persist longer than the encryption (eg. archival use)**
  - **MAC can be computed either before or after encryption**
    - different ways for combining the two functions
    - there are algorithms that integrate the two functions
      - authenticated encryption (AE)
- MAC is similar but not equal to digital signature

Hash Message Authenication Code (H-MAC)

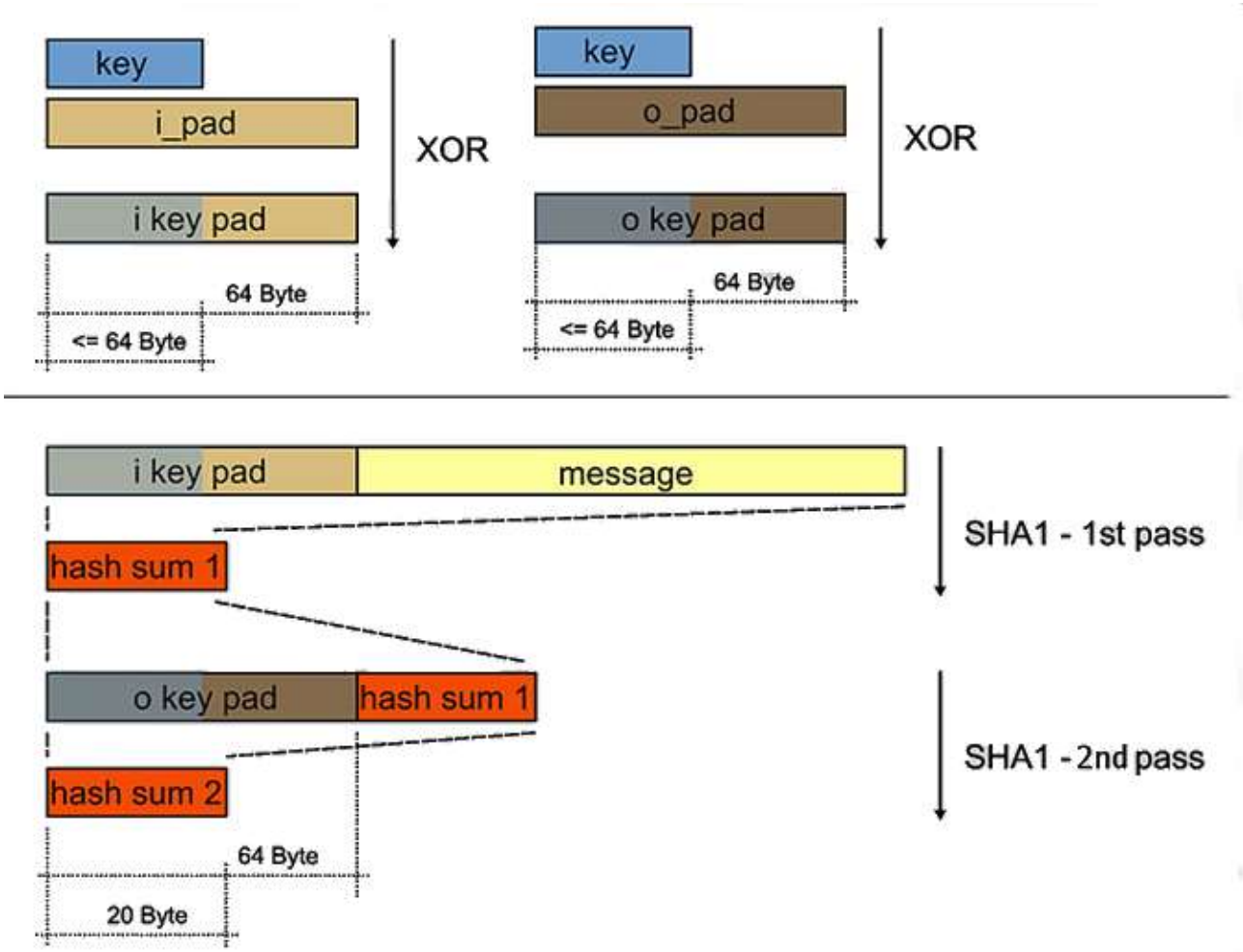
- Algorithm for creating a message authentication code based on a cryptographic hash function and a secret shared key
- Specified as Internet standard RFC2104
- HMAC can be used with any iterative cryptographic hash function, e.g., MD5, SHA-1, SHA-256, etc
  - **the cryptographic strength of H-MAC depends on the properties of the underlying hash function**

H-MAC (cont.)

- HMAC(K,m) = Hash[(K⁺ XOR opad) || Hash[(K⁺ XOR ipad)|| m)]]
  - **where K+ is the key 0-padded out to size b**
    - *b* is the size of the processing block
      - e.g. *b* = 512bits = 64bytes for SHA1
    - if *K* is longer than *b* bytes it is first hashed using H
  - **and opad, ipad are specified padding constants**
    - ipad = the byte 0x36 repeated *b/8* times
    - opad = the byte 0x5C repeated *b/8* times

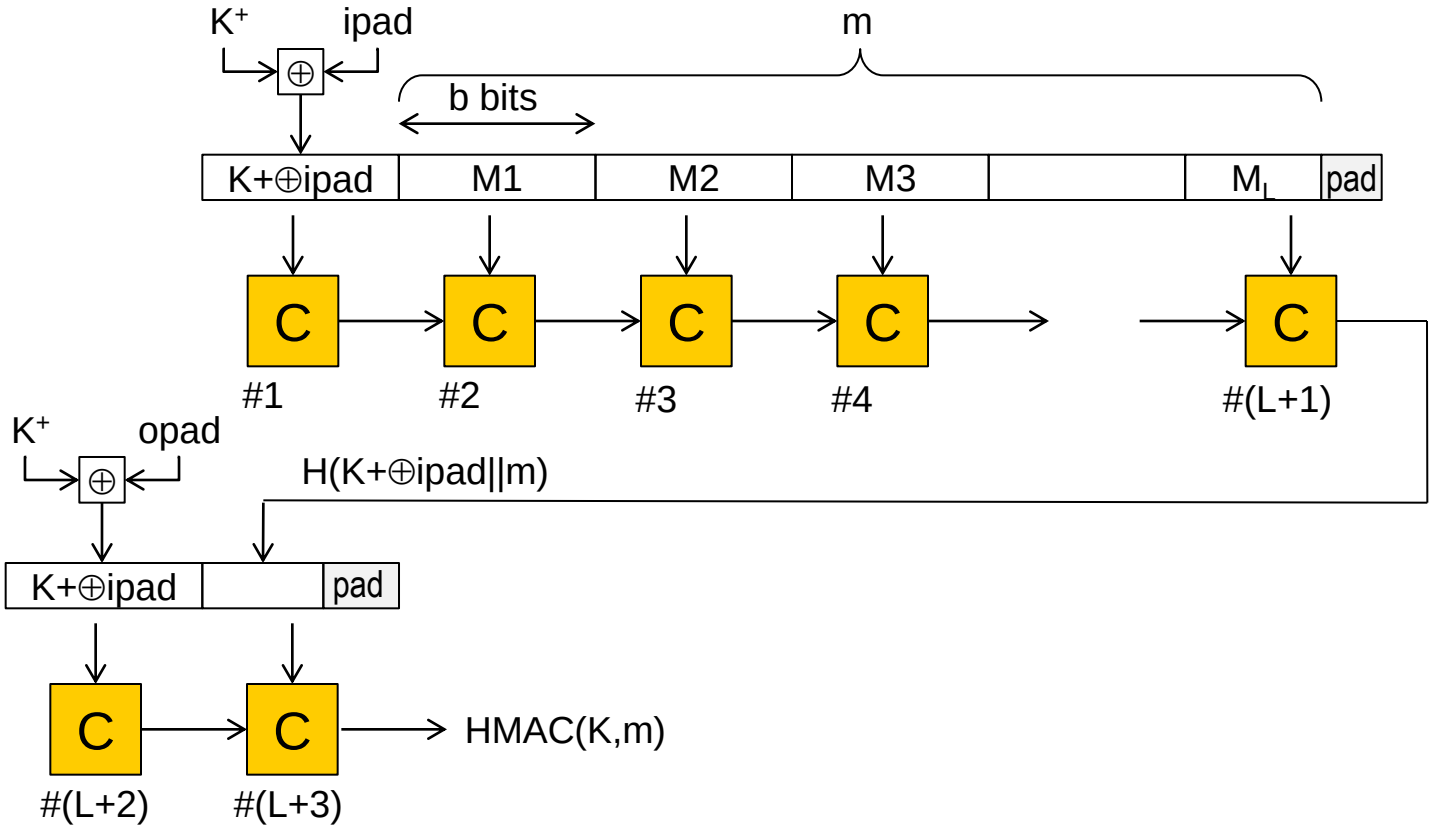


Example - H-MAC-SHA1



H-MAC (cont.)

- Overhead is just 3 more hash inner calculations (compression function *C*) than the message needs alone for computing message digest



Truncated H-MAC

- A well-known practice with MACs is to truncate the output of the MAC and output only part of the bits
  - **advantage: less information on the hash result available to an attacker**
  - **disadvantage: less bits to predict for the attacker**
- It is recommended to let the output length *t* be not less than half the length of the hash output and not less than 80 bits
- HMAC that uses a hash function *H* with *t* bits of output can be denoted as HMAC-*H*-*t*
  - **Example, HMAC-SHA1-80 denotes HMAC computed using the SHA-1 function and with the output truncated to 80 bits**

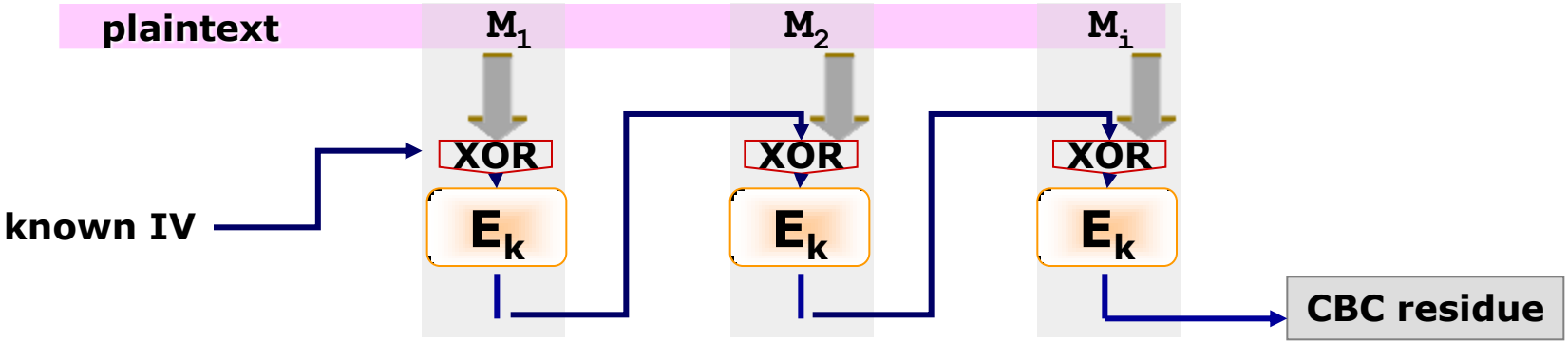
MAC Security

- Attacks:
  - **Cryptanalytic attacks**
  - **Brute-force attacks**

- Transient effect
  - **if the key is compromised or a the MAC scheme is broken, there are no effects on previously authenticated messages**
    - no adversarial effect on information authenticated in the past
  - **this is in contrast with encryption, where information encrypted today may suffer from exposure in the future when the encryption algorithm is broken or key is compromised**

Using Symmetric Ciphers for MAC

- If a cryptographic algorithm is available with CBC mode, a way of generating a MAC is to compute the CBC but keep only the last block (named CBC residue) as MAC value



- CBC-MAC standard mode use IV=0
- E.g. Data Authentication Algorithm (DAA) (now obsoleted) was a CBC-MAC based on DES
- Can use also other block cipher chaining modes and use final block

Using Symmetric Ciphers for MAC (cont.)

- Another approach could be to encrypt the hash
  - **$MAC_K(m) \equiv E_K(H(m))$**

- The main drawbacks of MAC functions based on symmetric ciphers are:
  - **lower speed (e.g. compared with HMAC)**
  - **size of the output may be too small for security (it depends on the block cipher)**

Authenticated Encryption

Authentication and Encryption

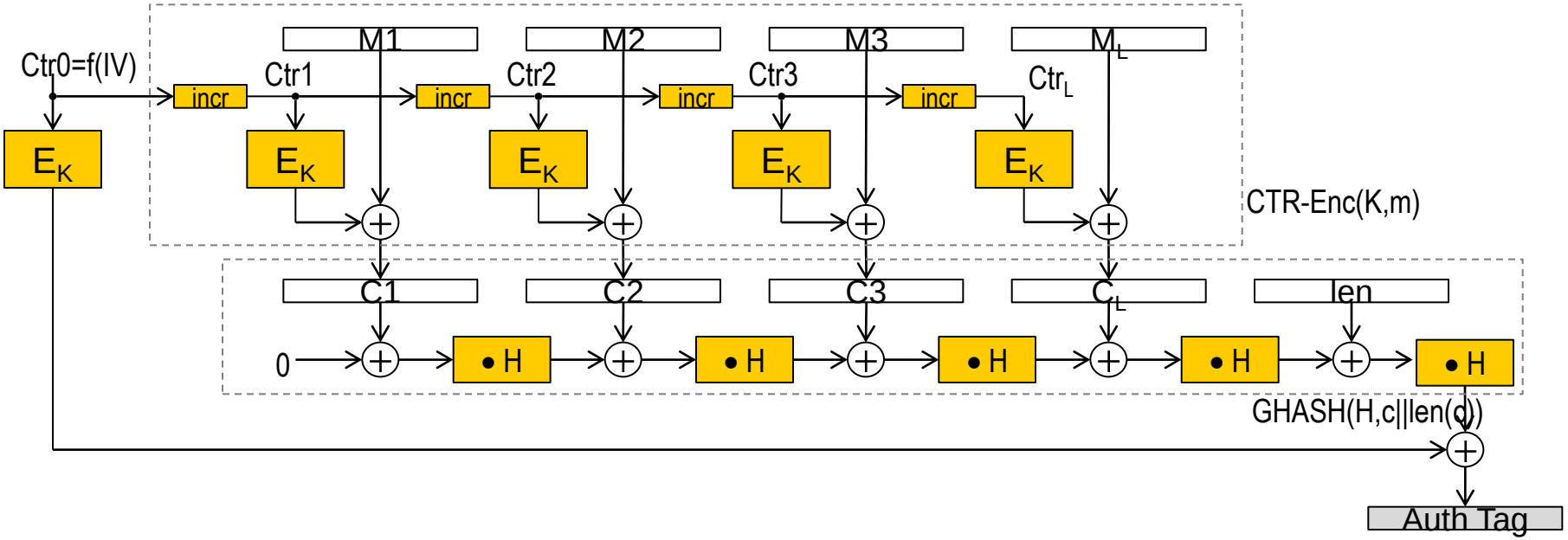
- Sometimes both message authentication and confidentiality are required
- There are four common approaches to providing both confidentiality and authenticity for a message *m* using two functions:
  - **Hash-then-Encrypt ***
    - $E_K(m||H(m))$
  - **MAC-then-Encrypt**
    - $E_{K2}(m|| MAC_{K1}(m))$
  - **Encrypt-then-MAC**
    - $C|| MAC_{K1}(C))$, where $C=E_{K2}(m)$
  - **Encrypt-and-MAC**
    - $E_{K2}(m) || MAC_{K1}(m)$
- Methods 2, 3, and 4 use two different keys

Authenticated Encryption

- Authenticated encryption (AE) is a cryptographic system that simultaneously protects confidentiality and authenticity (integrity)
- Example:
  - **CCM (Counter mode with CBC-MAC)**
    - it is a Encrypt-and-MAC scheme
    - ciphertext: c = CTR-Enc(K,m)
    - auth tag: t = Enc(K,Ctr0) ⊕ CBC-MAC(K,m)
      - *Ctr0* is the first generated counter based on a nonce value (then *Ctr1*,*Ctr2*,etc are used for encrypting)
- A more efficient design of an AE system is to process the message only one time (just one pass)
  - **instead of separately encrypting and computing the MAC**
  - **e.g. Galois/Counter Mode (GCM)**

Example: Galois/Counter Mode (GCM)

- It is Encrypt-and-MAC scheme:
  - **ciphertext: c= CTR-Enc(K,m), where Ctr0=f(IV)**
  - **auth tag: t= E(K,Ctr0)⊕GHASH(H,c||len(c))**
    - where GHASH(k,x) is a non-cryptographic keyed hash function
    - H= Enc(K,0)
    - X•Y is multiplication operation for the binary Galois field of 2^{128} elements
      - modular multiplication of binary polynomials of degree less than 128



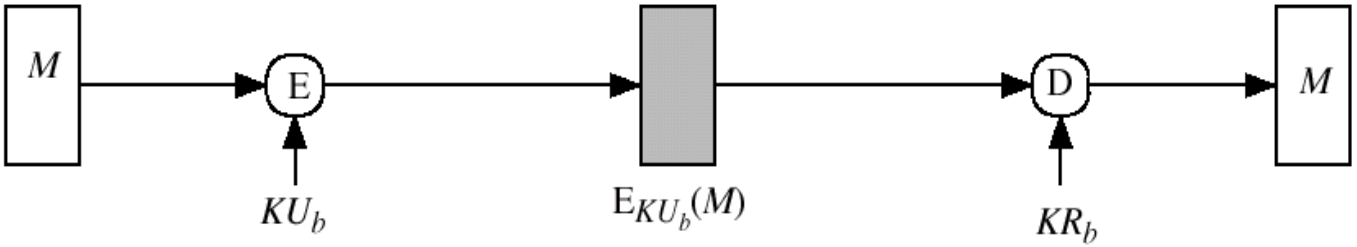
Authenticated Encryption with Associated Data

- Authenticated Encryption with Associated Data (AEAD)
  - **in addition to the plaintext *'m'* (that has to be AEed) there is extra data *'a'* that has to be only authenticated**
    - e.g. for protecting a network packet:
      - packet header (*A*) only authenticated since it must be readable,
      - packet payload (*m*) authenticated and encrypted
- Examples:
  - **CCM (Counter mode with CBC-MAC) AEAD**
    - c=CTR-Enc(K,m)
    - t=CBC-MAC(K,A||m) ⊕Enc(K,Ctr0)
  - **GCM (Galois/Counter Mode) AEAD**
    - c=CTR-Enc(K,m), where Ctr0=f(IV)
    - t=Enc(K,Ctr0)⊕GASH(H,A||c||len(A)||len(c)), with H=Enc(K,0)

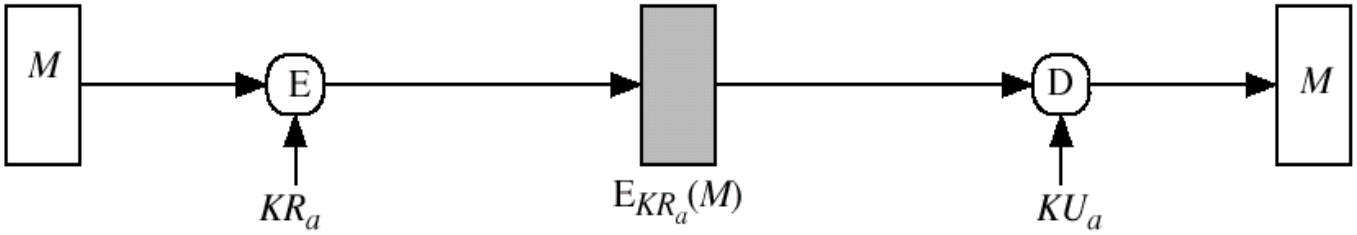
Digital Signature

Msg. Auth. - Asymmetric Encryption

- Asymmetric encryption with public key: confidentiality



- Asymmetric encryption with private key: authentication
  - **the resulting ciphertext can could be computed only by the owner of the private key**
    - a sort of signature
  - **however need to recognize corrupted messages**



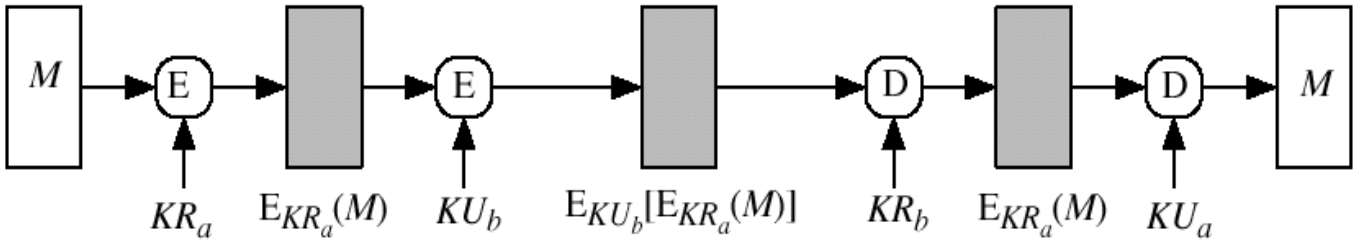
Msg. Auth. - Asymmetric Encryption (cont.)

- if public-key encryption is used
  - **encryption with public key provides no proof of sender (no sender authentication)**
    - since anyone potentially knows public-key
  - **encryption with private key provides authentication of the sender ("signature")**
    - if it is possible to distinguish, after decryption, between a valid message and a random string of bits

- Problems
  - **need to recognize corrupted messages**
  - **the signs has the same cost of public-key encryption of the entire message**

Msg. Auth. - Asymmetric Encryption

- If both confidentiality and authentication are required
  - **sender may "signs" message using their private-key**
  - **then encrypts with recipients public key**



Digital Signature

- A digital signature scheme (or mechanism) consists of
  - **a signature generation algorithm**
    - a method for producing a digital signature
  - **a signature verification algorithm**
    - a method for verifying that a digital signature is authentic (i.e., was indeed created by the specified entity)
- According to a a digital signature scheme
  - **a signer "signs" a message *m*, using her private key, in such a way that**
  - **anyone can "verify", using the corresponding public key, that the message was signed by no one other than actual signer**
    - consequently that the message has not been modified
      - i.e. the message is a true and correct copy of the original
- The difference between digital signatures and conventional ones is that digital signatures can be mathematically verified

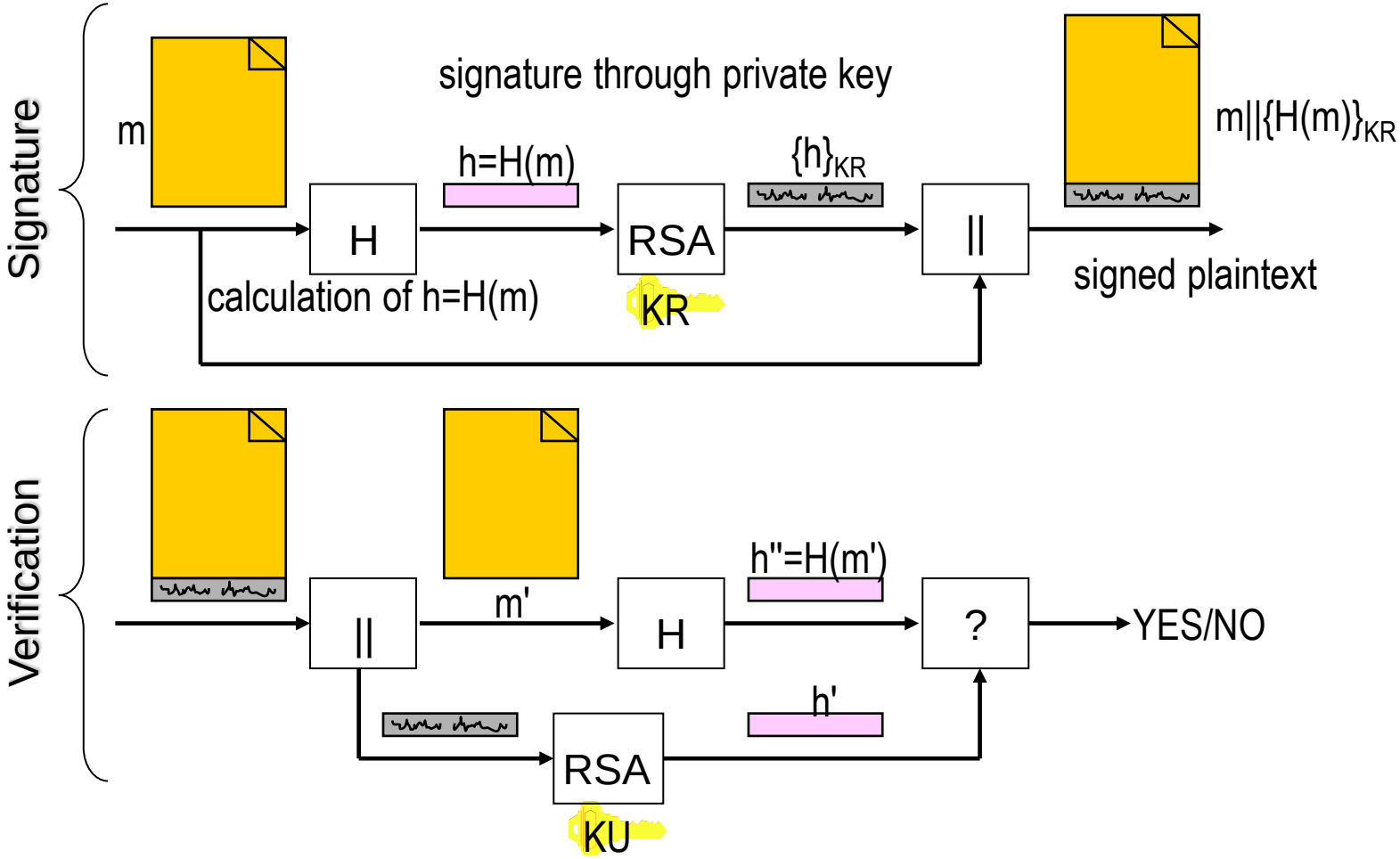
Digital Signature (cont.)

- Two general classes of digital signature schemes:
  - **Digital signature schemes with appendix (SSA)**
    - require the original message as input to the verification algorithm
  - **Digital signature schemes with message recovery**
    - do not require the original message as input to the verification algorithm
    - the original message is recovered from the signature itself
      - the signature contains the message

- A digital signature scheme is said to be:
  - **deterministic**
    - signing operation is a one message-to-signature transformation
  - **randomized**
    - if the signing operation is a function also of a second parameter, leading to an indexing set of message-to-signature transformations

RSA Digital Signature with Appendix

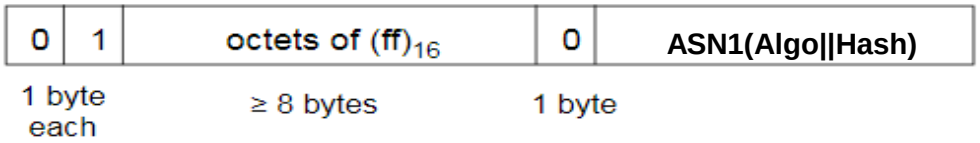
- RSA Signature Scheme with Appendix (RSA-SSA)
  - **H(m) = hash of padded m**



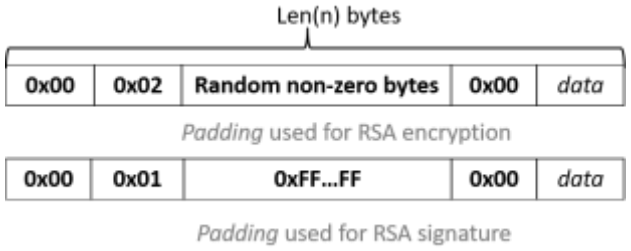
RSA Signature

- PKCS#1 "RSA Cryptography Specifications Version 2.2" (RFC 8017) specifies two encoding methods for signature scheme with appendix (SSA):
  - **RSASSA-PKCS1-v1_5**
    - uses deterministic encoding
  - **RSASSA-PSS (Probabilistic Signature Scheme)**
    - uses probabilistic encoding
      - includes a salt value
- These signature schemes combine signature/verification primitives with an encoding method for signatures
  - **a message encoding operation is applied to a message to produce an encoded message, which is then converted to an integer and processed by RSA signature primitive**
- Although no attacks are known against RSASSA-PKCS1-v1_5, RSASSA-PSS is preferred in new applications

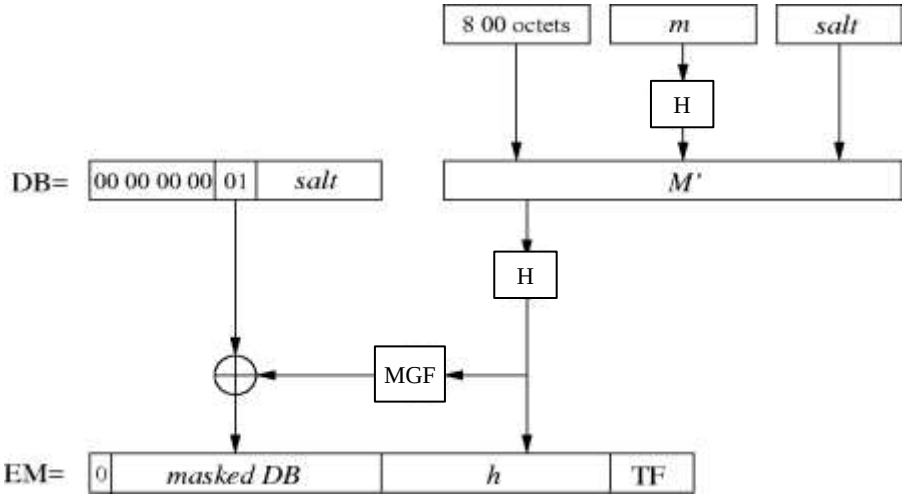
RSASSA-PKCS1-v1.5 encoding



- Deterministic encoding
  - **produce the hash value h = Hash(M)**
  - **produce T = ASN.1 DER encoding of the algorithm ID for the hash function and the hash value h**
    - the first field identifies the hash function and the second contains the hash value
  - **generate an octet string PS consisting of emLen - tLen - 3 octets with hexadecimal value 0xff, where tLen and emlen are the length of T and EM respectively**
    - the length of PS will be at least 8 octets
  - **concatenate PS, T, and other padding to form the encoded message EM**
    - EM= 0x00 || 0x01 || PS || 0x00 || T
- Comparison with the padding algorithm used for RSA encryption:



RSASSA-PSS encoding



- Probabilistic Signature Scheme
  - **M' = (0x)00 00 00 00 00 00 00 00 || mHash || salt**
    - salt = random octet string of length sLen
    - mHash = Hash(M), of length hLen
    - length of M' is 8 + hLen + sLen
  - **h = Hash(M'), octet string of length hLen**
  - **data block DB = PS || 0x01 || salt**
    - PS = zero octet string of length emLen - sLen - hLen - 2 (length may be 0)
    - length of DB is emLen - hLen - 1
  - **dbMask = MGF(H, emLen - hLen - 1), where MGF(seed,len) is the mask generation function**
    - MGF1(seed,len) = H(seed || 0x00000000) || H(seed || 0x00000001) || H(seed || 0x00000002) || H(seed || 0x00000003) || …
  - **maskedDB = DB xor dbMask**
  - **set the leftmost 8emLen - emBits bits of the leftmost octet in maskedDB to zero**
  - **EM = maskedDB || h || 0xbc**

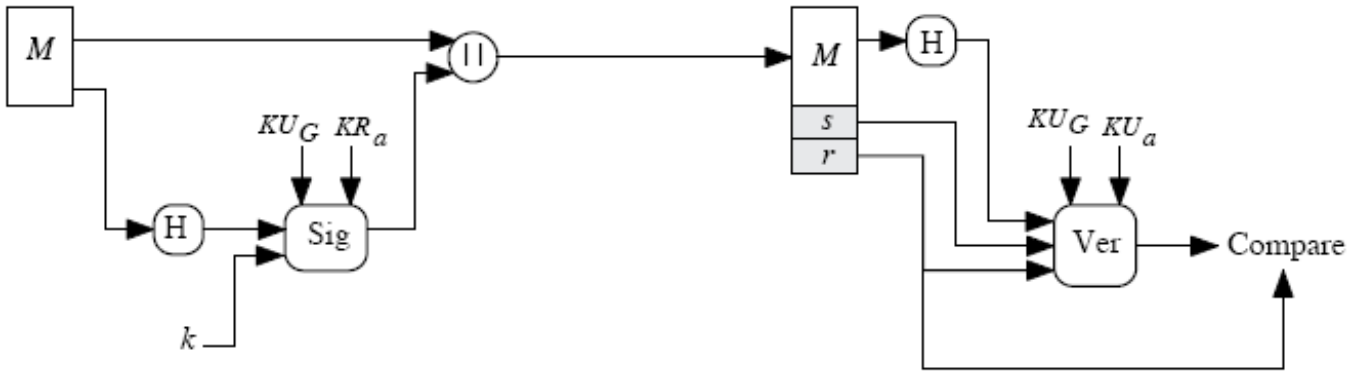
Possible RSA signature vulnerabilities

- The main issues related to RSA signature arise if the message is signed without hashing and/or proper padding
  - **e.g. using direct RSA exponentiation of the message** *m*

- In case of direct exponentiation, for example:
  - **some signature are independent from the value of** *d*:
    - $\text{sign}(0) = 0$
    - $\text{sign}(1) = 1$
    - $\text{sign}(n\text{-}1) = n\text{-}1$
  - **due to the homomorphic properties of RSA, in presence of a sign oracle it is possible to obtain the signature of an arbitrary message** *m* **by asking to sign a different message (blind signature):**
    - select a value r
    - ask to sign $r^e m$
      - $\text{sign}(r^e m) = (r^e m)^d \bmod n = r m^d \bmod n$
    - by multiplying the result by r^{-1} you obtain $m^d \bmod n = \text{sign}(m)$

Digital Signature Standard (DSS)

- DSS (Digital Signature Standard)
- Proposed by NIST (U.S. National Institute of Standards and Technology) & NSA in 1991
  - **FIPS 186**
- Based on an algorithm known as DSA (Digital Signature Algorithm)
  - **is a variant of the Elgamal (Taher Elgamal) scheme**
  - **uses hash algorithm, size *N* (e.g. SHA1, size 160)**
  - **uses *N*-bit exponents**
  - **creates a *2N* bit signature *(s,r)* but with *L=1024* (or more) bit security**



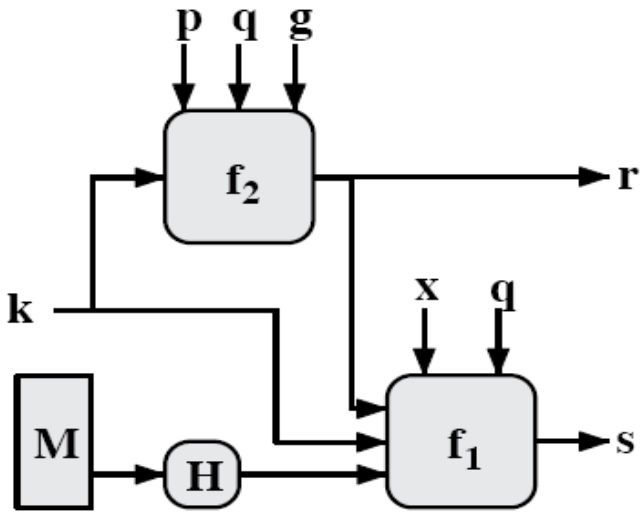
- Security depends on difficulty of computing discrete logarithms

DSA Key Generation

- have shared global public key values (p,q,g)
  - L **and** N **are respectively the key length and hash length**
    - L = 1024 or more, and multiple of 64 (e.g. 1024, 2048, 3072,..)
    - N = 160 or more (e.g. 160, 256, ..)
  - **take a large (**L**-bit) prime** p
  - **choose q, a** N**-bit prime factor of** $p-1$
    - in practice, you can choose q, and then p such that $(p-1)$ is multiple of q
  - **choose** g **such that its multiplicative order modulo** p **is** q
    - in practice, $g = a^{(p-1)/q} \bmod p$
      - for some arbitrary *a* with $1<a<p-1$, with $a^{(p-1)/q} \bmod p > 1$
- choose $x<q$
- compute $y = g^x \bmod p$
- public key = (p,q,g,y)
- private key = x

DSS Signing and Verifying overview

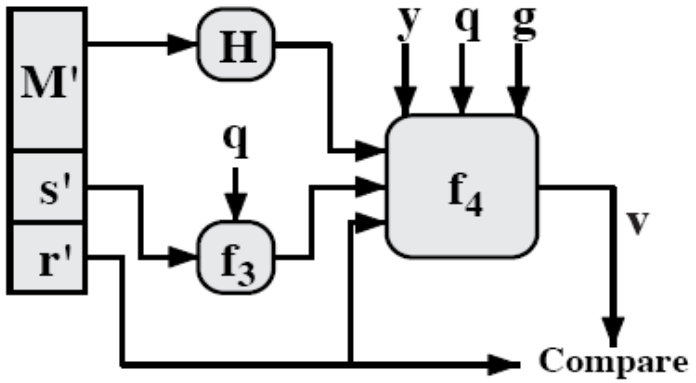
- Signing



$r = f2(k,p,q,g) = (g^k \bmod p) \bmod q$

$s = f1(H(m),k,x,r,q) = (k^{-1}(H(m)+xr)) \bmod q$

- Verifying



$w = f3(s,q) = s^{-1} \bmod q$

$v = f4(p,q,g,y,H(m),w,r) =$
$= ((g^{H(m)w \bmod q}\ y^{rw \bmod q}) \bmod p) \bmod q$

DSA Signature

- to sign a message m the sender generates:
  - **a random signature key** k, $k<q$
    - k must be random, be destroyed after use, and never be reused
- computes the message digest (e.g. SHA-1) of the message m:

  $h = H(m)$
- then computes signature pair:

  $r = (g^k \bmod p) \bmod q$

  $s = k^{-1}(h+x \cdot r) \bmod q, \quad \text{with } k \cdot k^{-1} \equiv 1 \pmod q$
- sends signature (r,s) with message m

DSA Signature Verification

- having received m & signature (r,s)
- to verify a signature, recipient computes:

$$w = s^{-1} \bmod q$$

$$v = (g^{hw \bmod q}\, y^{rw \bmod q} \bmod p) \bmod q$$

- if $v=r$ then signature is verified

- proof

$$v = (g^{hw \bmod q}\, y^{rw \bmod q} \bmod p) \bmod q =$$
$$= (g^{w(h+xr) \bmod q} \bmod p) \bmod q =$$
$$= (g^{k} \bmod p) \bmod q =$$
$$= r$$

Possible DSA vulnerabilities

- The main issue with DSA is related to the choice of the nonce k
  - **in general, using not random (or biased) nonces is a bad idea**
- If k is known or guessable
  - **since:**
    - $r = g^k \bmod p \bmod q$
    - $s = (k^{-1} (h + xr)) \bmod q$
  - **the private key x can be recovered as follows:**
    - $x = (sk - h)\, r^{-1}$
- If k is used more than once
  - **suppose the user signs two messages m_1, m_2 using the same nonce k**
    - (r_1, s_1) and (r_2, s_2) are the two signatures
  - **the private key x can be recovered as follows:**
    - $x = (s_2 h_1 - s_1 h_2)(r_2 s_1 - r_1 s_2)^{-1} \bmod q$

Possible DSA vulnerabilities (cont.)

- If k is incremented by a constant value
  - **suppose a user signs two times the same message m with the nonce values k_1 and $k_2 = k_1 + d$**
  - **defining $S=h+xr$ we have:**
    - $s_1 = k_1^{-1} S \bmod q$
    - $s_2 = (k_1 + d)^{-1} S \bmod q$
  - **that is**
    - $S = s_1 k_1 \bmod q$
    - $S = s_2(k_1 + d) \bmod q$
  - **then**
    - $s_1 k_1 \equiv s_2(k_1+d) \Rightarrow k_1(s_1-s_2) \equiv s_2 d \Rightarrow k_1 = s_2 d (s_1-s_2)^{-1} \bmod q$
  - **by knowing k_1 it is possible to recover the private key x**